Homework #7

3.1T4 3.1T5 3.2T5 3.3T1 3.3T3 3.4T1 3.4T7 附加题

3.1T4

4. 对于币值为 1, 3, 5 且 $n = 9$ 的找零问题，利用动态规划算法求其所有解。

解.

n	0	1	2	3	4	5	6	7	8	9
F	0	1	2	1	2	1	2	3	2	3



F[0]=0

F[1]= min{F[1-1]} +1 = 1

F[2]= min{F[2-1]} +1 = 2

F[3]= min{F[3-1], F[3-3]} +1 = 1

F[4]= min{F[4-1], F[4-3]} +1 = 2

F[5]= min{F[5-1], F[5-3]+F[5-5]} +1 = 1

F[6]= min{F[6-1], F[6-3]+F[6-5]} +1 = 2

F[7]= min{F[7-1], F[7-3]+F[7-5]} +1 = 3

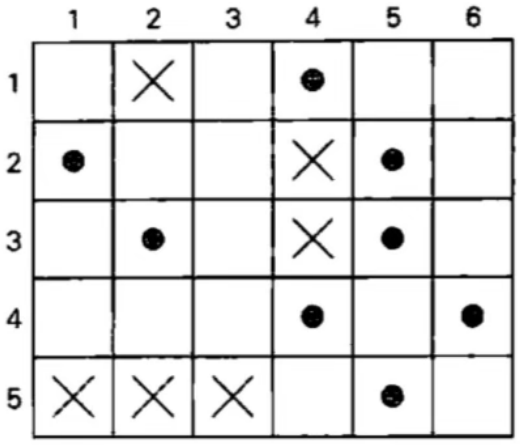
F[8]= min{F[8-1], F[8-3]+F[8-5]} +1 = 2

F[9]= min{F[9-1], F[9-3]+F[9-5]} +1 = 3

硬币组合为 1, 3, 5 或 3, 3, 3

8.1T5

5. 对于有些单元格不可达的硬币收集问题，如何修改原来的动态规划算法？应用新算法求解下图问题，其中不可达的单元格用×表示。对于这块板而言有多少条最优路径？



解：同样的，若单元格(i,j)中有硬币存在，

则 $c_{ij}=1$，否则 $c_{ij}=0$

但对于不可达单元格，$c_{ij}=-\infty$

且 $F(0,j)=-\infty$ $1\le j\le m$；$F(i,0)=-\infty$ $1\le i\le n$，$F(1,1)=0$

则

	1	2	3	4	5	6
1	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
2	1	1	1	$-\infty$	$-\infty$	$-\infty$
3	1	2	2	$-\infty$	$-\infty$	$-\infty$
4	1	2	2	3	3	4
5	$-\infty$	$-\infty$	$-\infty$	3	4	4

路径数 $=2\times2\times3=12$种

8.2T5

5. 假设 n 种物品中每种物品的数量不限，为该背包问题设计一个动态规划算法并分析该算法的时间效率。

解：若n种物品中每种物品数量不限，只需修改递推式

$$F(i,j)=\begin{cases}\max\{F(i-1,j),\ v_i+F(i,j-w_i)\} & j\geq w_i\\ F(i-1,j) & j<w_i\end{cases}$$

即物品 i 用过以后仍可继续用，时间效率 $\Theta(nW)$

或者直接类似于找零问题

$$F(W)=\begin{cases}\max\limits_{j:W\geq w_j}\{F(W-w_j)+v_j\}\\ 0 \quad w_j>W\ (j=1,2,\cdots n)\end{cases}$$

时间效率 $\Theta(nW)$

8.3T1

1. 完成本节构造最优二叉查找树的例题中余下的计算。

例 让我们对本节开头使用的 4 个键的集合应用该算法，来给大家一个感性的认识。

键	A	B	C	D
查找概率	0.1	0.2	0.4	0.3

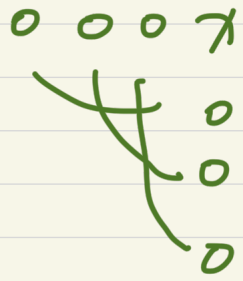
初始表格如下：

主表

	0	1	2	3	4
1	0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	1.4
3			0	0.4	1
4				0	0.3
5					0

根表

	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					



我们来计算 $C(1,2)$：

$$C(1,2)=\min\begin{cases}k=1:\ C(1,0)+C(2,2)+\sum_{s=1}^{2}p_s=0+0.2+0.3=0.5\\ k=2:\ C(1,1)+C(3,2)+\sum_{s=1}^{2}p_s=0.1+0+0.3=0.4\end{cases}=0.4$$

解：$C(i,j)=\min_{i<k\le j}\{C(i,k-1)+C(k,j)\}+\sum_{s=i}^{j}P_s$

$$C(2,3)=\min\begin{cases}k=2 & C(2,1)+C(3,3)=0.4\\ k=3 & C(2,2)+C(4,3)=0.2\end{cases}+\sum_{s=2}^{3}P_s=0.2+0.6=0.8$$

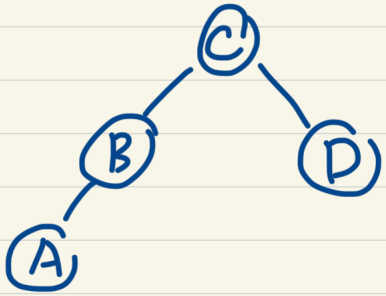
$$C(3,4)=\min\begin{cases}k=3 & C(3,2)+C(4,4)=0.3\\ k=4 & C(3,3)+C(5,4)=0.4\end{cases}+\sum_{s=3}^{4}P_s=0.3+0.7=1$$

$$C(1,3)=\min\begin{cases}k=1 & C(1,0)+C(2,3)=0.8\\ k=2 & C(1,1)+C(3,3)=0.5\\ k=3 & C(1,2)+C(4,3)=0.4\end{cases}+\sum_{s=1}^{3}P_s=0.4+0.7=1.1$$

$$C(2,4)=\min\begin{cases}k=2 & C(2,1)+C(3,4)=1\\ k=3 & C(2,2)+C(4,4)=0.5\\ k=4 & C(2,3)+C(5,4)=0.8\end{cases}+\sum_{s=2}^{4}P_s=0.5+0.9=1.4$$

$$C(1,4)=\min\begin{cases}k=1 & C(1,0)+C(2,4)=1.4\\ k=2 & C(1,1)+C(3,4)=1.1\\ k=3 & C(1,2)+C(4,4)=a\\ k=4 & C(1,3)+C(5,4)=1.1\end{cases}+\sum_{s=1}^{4}P_s=0.7+1=1.7$$

最优二叉树



8.3T3

3. 写一个线性时间算法的伪代码，来从根表中生成最优二叉查找树。

```
Treemaking(i,j)
if i>j
    return NULL
else
    k←R[i,j]
    root← NewNode(Keys[k])
    root.left←Treemaking(i,k-1)
    root.right←Treemaking(k+1,j)
    return root
```

8.4T1

1. 对由下列邻接矩阵定义的有向图应用 Warshall 算法，求它的传递闭包。

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

解：

$R^{(0)}=$

	a	b	c	d
a	0	1	0	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

$R^{(1)}=$

	a	b	c	d
a	0	1	0	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

$R^{(2)}=$

	a	b	c	d
a	0	1	1	0
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

$R^{(3)}=$

	a	b	c	d
a	0	1	1	1
b	0	0	1	1
c	0	0	0	1
d	0	0	0	0

$R^{(4)}=$

	a	b	c	d
a	0	1	1	1
b	0	0	1	1
c	0	0	0	1
d	0	0	0	0

8.4T7

7. 对于下面具有权重矩阵的有向图，求解完全最短路径问题。

$$\begin{bmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & \infty \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & \infty & \infty & \infty & 0 \end{bmatrix}$$

解.

$D^{(0)}=$

	a	b	c	d	e
a	0	2	∞	1	8
b	6	0	3	2	∞
c	∞	∞	0	4	∞
d	∞	∞	2	0	3
e	3	∞	∞	∞	0

$D^{(1)}=$

	a	b	c	d	e
a	0	2	∞	1	8
b	6	0	3	2	14
c	∞	∞	0	4	∞
d	∞	∞	2	0	3
e	3	5	∞	4	0

$D^{(2)}=$

	a	b	c	d	e
a	0	2	5	1	8
b	6	0	3	2	14
c	∞	∞	0	4	∞
d	∞	∞	2	0	3
e	3	5	8	4	0

$D^{(3)}=$

	a	b	c	d	e
a	0	2	5	1	8
b	6	0	3	2	14
c	∞	∞	0	4	∞
d	∞	∞	2	0	3
e	3	5	8	4	0

$D^{(4)}=$

	a	b	c	d	e
a	0	2	3	1	4
b	6	0	3	2	5
c	∞	∞	0	4	7
d	∞	∞	2	0	3
e	3	5	6	4	0

$D^{(5)}=$

	a	b	c	d	e
a	0	2	3	1	4
b	6	0	3	2	5
c	10	12	0	4	7
d	6	8	2	0	3
e	3	5	6	4	0

算法设计题

五、（12 分）给定一个正整数 n，现需设计一个高效的算法，找出最少个数的完全平方数（如 1，4，9，16,...），使得这些完全平方数的和等于n。↵

（1）（10 分）简要说明所设计算法的策略或思想，并用伪代码描述算法。↵

（2）（2 分）分析所设计算法的时间复杂度。↵

举例：输入 $n = 12$，输出为 3（因为 $12 = 4+4+4$）;↵

输入 $n = 13$，输出为 2（因为 $13 = 4+9$）。↵

解 (1) 动态规划

```
SquareFinding(n)
F[0]←0
for i←1 to n do
    temp←∞; j=1
    while i≥j² do
        temp←min(F[i-j²], temp)
        j=j+1
    F[i]←temp+1
return F[n]
```

(2) 时间效率 $\Theta(n^{\frac{3}{2}})$

空间效率 $\Theta(n)$